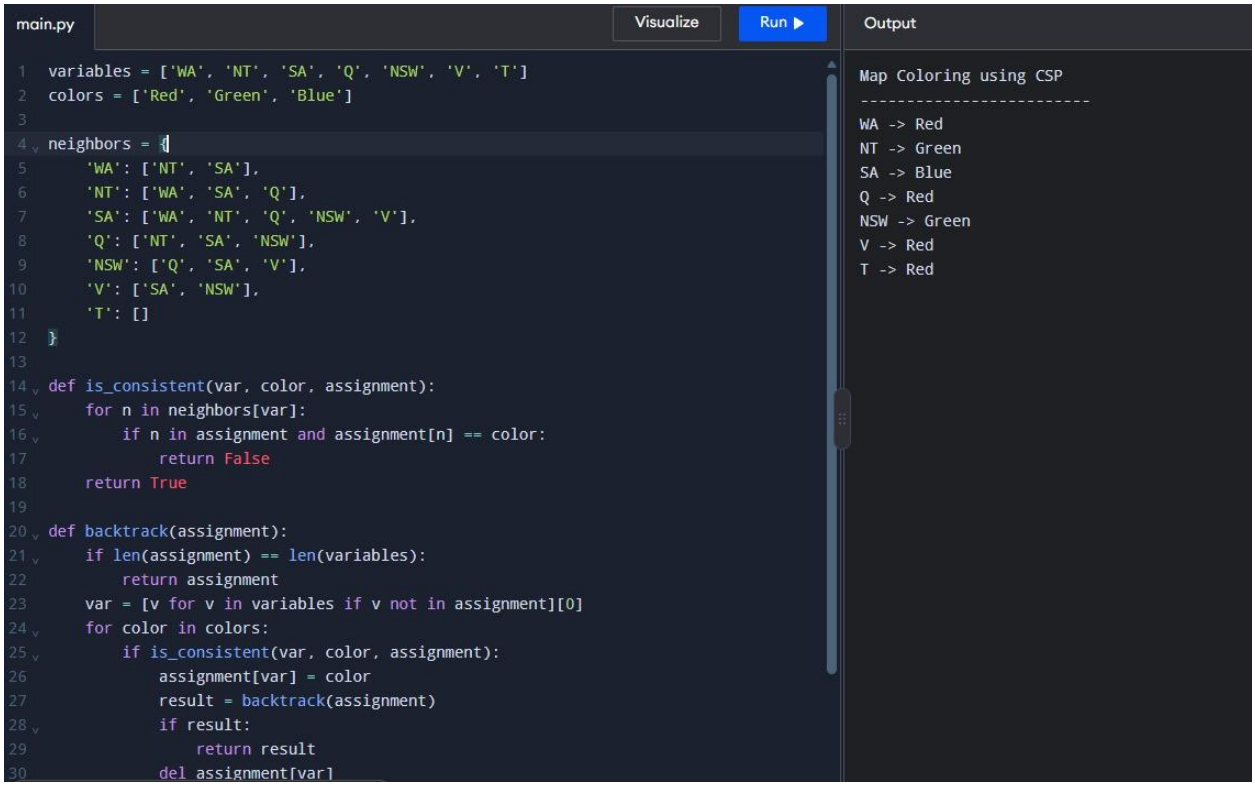
1. **Write the python program for Map Coloring to implement CSP.**




```
variables = ['WA', 'NT', 'SA', 'Q', 'NSW', 'V', 'T']
colors = ['Red', 'Green', 'Blue']

neighbors = {
    'WA': ['NT', 'SA'],
    'NT': ['WA', 'SA', 'Q'],
    'SA': ['WA', 'NT', 'Q', 'NSW', 'V'],
    'Q': ['NT', 'SA', 'NSW'],
    'NSW': ['Q', 'SA', 'V'],
    'V': ['SA', 'NSW'],
    'T': []
}

def is_consistent(var, color, assignment):
    for n in neighbors[var]:
        if n in assignment and assignment[n] == color:
            return False
    return True

def backtrack(assignment):
    if len(assignment) == len(variables):
        return assignment
    var = [v for v in variables if v not in assignment][0]
    for color in colors:
        if is_consistent(var, color, assignment):
            assignment[var] = color
            result = backtrack(assignment)
            if result:
                return result
            del assignment[var]
    return None

solution = backtrack({})
print("Map Coloring using CSP")
print("-" * 25)
if solution:
    for region, color in solution.items():
        print(region, "->", color)
else:
    print("No solution found")
```

12. **Write the python program for Tic Tac Toe game.**

main.py Visualize Run ▶

```
# Tic Tac Toe Game (2 players)

board = [" "] * 9

def show_board():
    print()
    print(" " + board[0] + " | " + board[1] + " | " + board[2])
    print("---+---+---")
    print(" " + board[3] + " | " + board[4] + " | " + board[5])
    print("---+---+---")
    print(" " + board[6] + " | " + board[7] + " | " + board[8])
    print()

def check_winner():
    wins = [(0, 1, 2), (3, 4, 5), (6, 7, 8),
            (0, 3, 6), (1, 4, 7), (2, 5, 8),
            (0, 4, 8), (2, 4, 6)]
    for a, b, c in wins:
        if board[a] == board[b] == board[c] and board[a] != " ":
            return board[a]
    return None

player = "X"
```

```
print("Tic Tac Toe Game")
print("Positions 1 to 9 use pannunga:")
print(" 1 | 2 | 3 ")
print(" 4 | 5 | 6 ")
print(" 7 | 8 | 9 ")

for turn in range(9):
    show_board()
    while True:
        move = input("Player " + player + " - position (1-9): ")
        if move.isdigit() and 1 <= int(move) <= 9 and board[int(move) - 1] == " ":
            board[int(move) - 1] = player
            break
        print("Thappu move! Veroru position kudunga.")

    if check_winner():
        show_board()
        print("Player " + player + " wins!")
        break

    if player == "X":
        player = "O"
    else:
        player = "X"
else:
```

```
Tic Tac Toe Game
Positions 1 to 9 use pannunga:
 1 | 2 | 3
 4 | 5 | 6
 7 | 8 | 9

   |   |
---+---+---
   |   |
---+---+---
   |   |

Player X - position (1-9):

=== Session Ended Due To Inactivity ===
```

```
            print("Player " + player + " wins!")
            break

    if player == "X":
        player = "O"
    else:
        player = "X"
else:
    show_board()
    print("Match Draw!")
```

13. Write the python program to implement Minimax algorithm for gaming

main.py Visualize Run ▶

```python
## Minimax - Tic Tac Toe (You = X, Computer = O)
b = [" "] * 9
W = [(0,1,2),(3,4,5),(6,7,8),(0,3,6),(1,4,7),(2,5,8),
(0,4,8),(2,4,6)]

def win():
    for x, y, z in W:
        if b[x] == b[y] == b[z] != " ":
            return b[x]

def show():
    for i in (0, 3, 6):
        print(" | ".join(b[i:i+3]))
    print()

def minimax(is_max):
    w = win()
    if w: return 1 if w == "O" else -1
    if " " not in b: return 0
    scores = []
    for i in range(9):
        if b[i] == " ":
            b[i] = "O" if is_max else "X"
            scores.append(minimax(not is_max))
            b[i] = " "
    return max(scores) if is_max else min(scores)

```

Output

```
  |   |
  |   |
  |   |

Your move (1-9):
```

```
26
27  def best_move():
28      best, move = -2, 0
29      for i in range(9):
30          if b[i] == " ":
31              b[i] = "O"
32              s = minimax(False)
33              b[i] = " "
34              if s > best: best, move = s, i
35      return move
36
37  while True:
38      show()
39      m = input("Your move (1-9): ")
40      if not (m.isdigit() and 1 <= int(m) <= 9 and
    b[int(m)-1] == " "):
41          print("Invalid move"); continue
42      b[int(m)-1] = "X"
43      if not win() and " " in b:
44          b[best_move()] = "O"
45      if win() or " " not in b:
46          show()
47          print(win() + " wins!" if win() else "Draw!")
48          break
```

14. Write the python program to implement Alpha & Beta pruning algorithm for gaming.

main.py Visualize Run ▶ Output

```python
# Alpha-Beta Pruning
import math

# Leaf values of a game tree (depth 3, branching factor 2)
values = [3, 5, 6, 9, 1, 2, 0, -1]

def alphabeta(depth, index, is_max, alpha, beta):
    # Leaf node
    if depth == 3:
        return values[index]

    if is_max:
        best = -math.inf
        for i in range(2):
            val = alphabeta(depth + 1, index * 2 + i,
False, alpha, beta)
            best = max(best, val)
            alpha = max(alpha, best)
            if beta <= alpha:
                print("Pruned at depth", depth, "(alpha
=", alpha, ", beta =", beta, ")")
                break
        return best
    else:
        best = math.inf
        for i in range(2):
            val = alphabeta(depth + 1, index * 2 + i,
```

```
Leaf values: [3, 5, 6, 9, 1, 2, 0, -1]
Pruned at depth 2 (alpha = 6 , beta = 5 )
Pruned at depth 1 (alpha = 5 , beta = 2 )
Optimal value is: 5
```

```python
            val = alphabeta(depth + 1, index * 2 + i,
True, alpha, beta)
            best = min(best, val)
            beta = min(beta, best)
            if beta <= alpha:
                print("Pruned at depth", depth, "(alpha
=", alpha, ", beta =", beta, ")")
                break
        return best

print("Leaf values:", values)
print("Optimal value is:", alphabeta(0, 0, True, -
math.inf, math.inf))
```

15. Write the python program to implement Decision Tree

main.py Visualize Run ▶

```python
# Decision Tree (ID3 algorithm) - Play Tennis
import math
from collections import Counter

# Outlook, Temp, Humidity, Wind, Play
data = [
    ["Sunny", "Hot", "High", "Weak", "No"],
    ["Sunny", "Hot", "High", "Strong", "No"],
    ["Overcast", "Hot", "High", "Weak", "Yes"],
    ["Rain", "Mild", "High", "Weak", "Yes"],
    ["Rain", "Cool", "Normal", "Weak", "Yes"],
    ["Rain", "Cool", "Normal", "Strong", "No"],
    ["Overcast", "Cool", "Normal", "Strong", "Yes"],
    ["Sunny", "Mild", "High", "Weak", "No"],
    ["Sunny", "Cool", "Normal", "Weak", "Yes"],
    ["Rain", "Mild", "Normal", "Weak", "Yes"],
    ["Sunny", "Mild", "Normal", "Strong", "Yes"],
    ["Overcast", "Mild", "High", "Strong", "Yes"],
    ["Overcast", "Hot", "Normal", "Weak", "Yes"],
    ["Rain", "Mild", "High", "Strong", "No"],
]
features = ["Outlook", "Temp", "Humidity", "Wind"]

def entropy(rows):
    counts = Counter(r[-1] for r in rows)
    total = len(rows)
    return -sum((c / total) * math.log2(c / total) for c
```

Output

```
Decision Tree:
Outlook = Overcast
    -> Yes
Outlook = Sunny
    Humidity = High
        -> No
    Humidity = Normal
        -> Yes
Outlook = Rain
    Wind = Strong
        -> No
    Wind = Weak
        -> Yes

Test sample: {'Outlook': 'Sunny', 'Temp': 'Cool', 'Humidity':
'High', 'Wind': 'Strong'}
Prediction (Play Tennis): No
```

```
28
29  def gain(rows, col):
30      total = len(rows)
31      result = entropy(rows)
32      for v in set(r[col] for r in rows):
33          subset = [r for r in rows if r[col] == v]
34          result -= (len(subset) / total) * entropy(subset)
35      return result
36
37  def build(rows, cols):
38      labels = set(r[-1] for r in rows)
39      if len(labels) == 1:
40          return labels.pop()
41      if not cols:
42          return Counter(r[-1] for r in rows).most_common(1)
    [0][0]
43      best = max(cols, key=lambda c: gain(rows, c))
44      tree = {features[best]: {}}
45      for v in set(r[best] for r in rows):
46          subset = [r for r in rows if r[best] == v]
47          tree[features[best]][v] = build(subset, [c for c
    in cols if c != best])
48      return tree
49
50  def show(tree, indent=""):
51      if not isinstance(tree, dict):
52          print(indent + "->", tree)
```

```
52          print(indent + "->", tree)
53          return
54      for feature, branches in tree.items():
55          for value, sub in branches.items():
56              print(indent + feature + " = " + value)
57              show(sub, indent + "    ")
58
59  def predict(tree, sample):
60      while isinstance(tree, dict):
61          feature = next(iter(tree))
62          tree = tree[feature][sample[feature]]
63      return tree
64
65  tree = build(data, [0, 1, 2, 3])
66  print("Decision Tree:")
67  show(tree)
68
69  test = {"Outlook": "Sunny", "Temp": "Cool", "Humidity":
    "High", "Wind": "Strong"}
70  print("\nTest sample:", test)
71  print("Prediction (Play Tennis):", predict(tree, test))
```